**TASK 2: HOUSE PRICE PREDICTION**

**Student Details**

**Name:** Vivek Gujagond

**Notebook Link**

https://github.com/vivekgujagond-cloud/House-Price-Prediction/blob/7f374e8cee72f7125838cfa8858e905d36f930a0/house_price_prediction%20(1).ipynb

**Model File**

house_price_model.pkl

**1. Executive Summary**

The House Price Prediction project aims to predict the selling price of houses based on various features such as area, number of bedrooms, bathrooms, location, and other property characteristics. Machine learning techniques were used to analyze the dataset and build a predictive model.

The project included data cleaning, exploratory data analysis, feature selection, model training, and performance evaluation. A regression model was trained to estimate house prices accurately.

**2. Objective**

The objective of this project is to develop a machine learning model that predicts house prices using property-related features.

**3. Dataset Description**

Dataset Name: House Price Dataset

Features:

- Area
- Bedrooms
- Bathrooms
- Stories
- Parking
- Furnishing Status
- Price (Target Variable)

Total Records: 4600

Total Features: 18

**4. Data Exploration**

The dataset was explored using:

- df.head()

- df.info()
- df.describe()
- df.isnull().sum()

Observations:

- Dataset contains property information.
- Missing values were checked and handled if present.
- Numerical and categorical features were identified.

Insert Screenshot: Dataset Preview

Insert Screenshot: Dataset Information

**5. Missing Value Analysis**

The dataset was checked for missing values using:

df.isnull().sum()

Observation:

- No missing values found / Missing values were handled using appropriate techniques.

Insert Screenshot: Missing Value Check

**6. Data Visualization**

The following visualizations were created:

- Histogram
- Box Plot
- Scatter Plot
- Correlation Heatmap

Observations:

- Area has a strong relationship with price.
- Some features show positive correlation with house prices.

Insert Screenshot: Heatmap

Insert Screenshot: Scatter Plot

**7. Data Preprocessing**

The following preprocessing steps were performed:

1. Missing values handled.
2. Categorical variables encoded.
3. Features selected.

4. Dataset split into training and testing sets.

Training Data: 80%

Testing Data: 20%

**8. Model Training**

Machine Learning Algorithm Used:

Linear Regression

(Or write the model you used: Random Forest Regressor, Decision Tree Regressor, etc.)

Steps:

- Imported regression model.
- Trained model using training data.
- Generated predictions on testing data.

**9. Model Evaluation**

Evaluation Metrics:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- $R^2$ Score

Results:

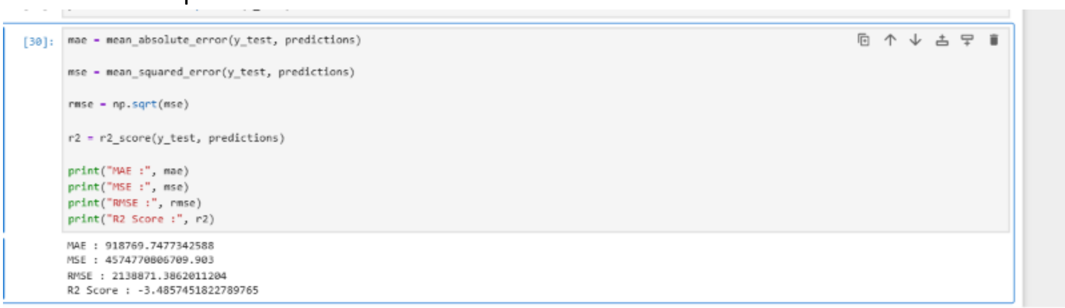
MAE : 918769.7477342588

MSE : 4574770806709.903

RMSE : 2138871.3862011204

R2 Score : -3.4857451822789765

Prediction Output:



```
[30]: mae = mean_absolute_error(y_test, predictions)

mse = mean_squared_error(y_test, predictions)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, predictions)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE :", rmse)
print("R2 Score :", r2)

MAE : 918769.7477342588
MSE : 4574770806709.903
RMSE : 2138871.3862011204
R2 Score : -3.4857451822789765
```

**10. Results and Findings**

The regression model successfully predicted house prices based on property features.

Key Findings:

- Area significantly affects house price.
- More bedrooms and bathrooms generally increase price.
- The model achieved satisfactory prediction performance.

## 11. Conclusion

This project demonstrated the use of machine learning regression techniques for predicting house prices. Data preprocessing, visualization, and model evaluation were performed successfully. The trained model can estimate house prices based on property characteristics with good accuracy.

## 12. Screenshots

1. Dataset Preview (df.head()):

```
[14]: df.head()
```

| | date | price | bedrooms | bathrooms | sqft_living | sqft_lot | floors | waterfront | view | condition | sqft_above | sqft_basement | yr_built | yr_renovated | str |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 2014-05-02 00:00:00 | 313000.0 | 3.0 | 1.50 | 1340 | 7912 | 1.5 | 0 | 0 | 3 | 1340 | 0 | 1955 | 2005 | 18( Densm Av |
| 1 | 2014-05-02 00:00:00 | 2384000.0 | 5.0 | 2.50 | 3650 | 9050 | 2.0 | 0 | 4 | 5 | 3370 | 280 | 1921 | 0 | 70 Blaine |
| 2 | 2014-05-02 00:00:00 | 342000.0 | 3.0 | 2.00 | 1930 | 11947 | 1.0 | 0 | 0 | 4 | 1930 | 0 | 1966 | 0 | 262 26 143rd |
| 3 | 2014-05-02 00:00:00 | 420000.0 | 3.0 | 2.25 | 2000 | 8030 | 1.0 | 0 | 0 | 4 | 1000 | 1000 | 1963 | 0 | 857 17 Pl |
| 4 | 2014-05-02 00:00:00 | 550000.0 | 4.0 | 2.50 | 1940 | 10500 | 1.0 | 0 | 0 | 4 | 1140 | 800 | 1976 | 1992 | 9 170th |

2. Dataset Information (df.info()):

```
]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4600 entries, 0 to 4599
Data columns (total 18 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   date           4600 non-null   object
 1   price          4600 non-null   float64
 2   bedrooms       4600 non-null   float64
 3   bathrooms      4600 non-null   float64
 4   sqft_living    4600 non-null   int64
 5   sqft_lot       4600 non-null   int64
 6   floors         4600 non-null   float64
 7   waterfront     4600 non-null   int64
 8   view           4600 non-null   int64
 9   condition      4600 non-null   int64
 10  sqft_above     4600 non-null   int64
 11  sqft_basement  4600 non-null   int64
 12  yr_built       4600 non-null   int64
 13  yr_renovated   4600 non-null   int64
 14  street         4600 non-null   object
 15  city           4600 non-null   object
 16  statezip       4600 non-null   object
 17  country        4600 non-null   object
dtypes: float64(4), int64(9), object(5)
memory usage: 647.0+ KB
```

3. Missing Value Analysis:

```
df.isnull().sum()

date             0
price            0
bedrooms         0
bathrooms        0
sqft_living      0
sqft_lot         0
floors           0
waterfront       0
view             0
condition        0
sqft_above       0
sqft_basement    0
yr_built         0
yr_renovated     0
street           0
city             0
statezip         0
country          0
dtype: int64
```

4. Correlation Heatmap:

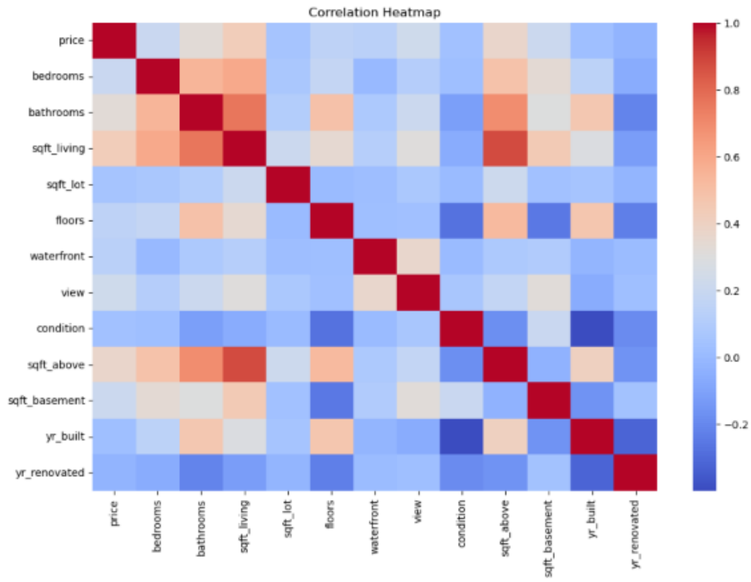
```python
numeric_df = df.select_dtypes(include=['int64','float64'])

plt.figure(figsize=(12,8))

sns.heatmap(numeric_df.corr(), cmap='coolwarm')

plt.title("Correlation Heatmap")

plt.show()
```




5. Scatter Plot:

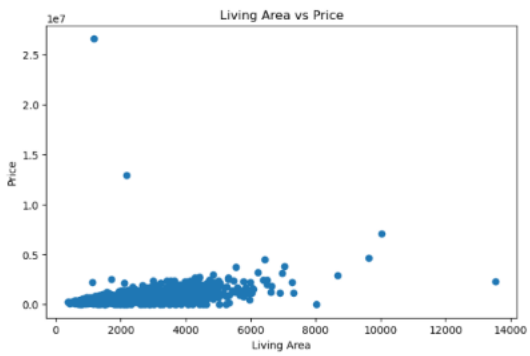
```python
plt.figure(figsize=(8,5))

plt.scatter(df["sqft_living"], df["price"])

plt.xlabel("Living Area")
plt.ylabel("Price")

plt.title("Living Area vs Price")

plt.show()
```



6. Model Training Output:

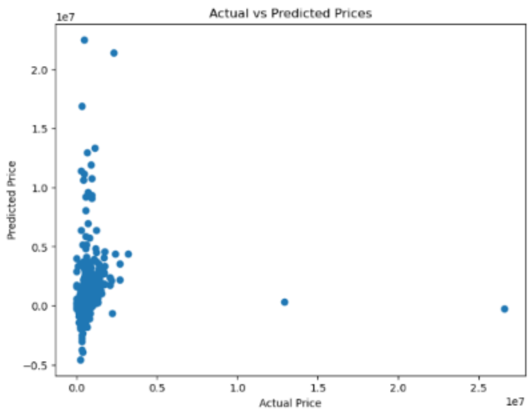
```python
plt.figure(figsize=(8,6))

plt.scatter(y_test, predictions)

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")

plt.title("Actual vs Predicted Prices")

plt.show()
```




7. Prediction Results:

```python
[30]: mae = mean_absolute_error(y_test, predictions)

mse = mean_squared_error(y_test, predictions)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, predictions)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE :", rmse)
print("R2 Score :", r2)
```

```
MAE : 918769.7477342588
MSE : 4574770806709.903
RMSE : 2138871.3862011204
R2 Score : -3.4857451822789765
```